

The background of the entire image is a dark blue field filled with a pattern of red dots. These dots are arranged in a way that they form a large, faint circular shape in the center, with the density of the dots being higher in the center and fading out towards the edges. The dots are of varying sizes, creating a textured, digital effect.

HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.



ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

HDSHELL: TINY SHELL

Project 1 – Môn Nguyên Lý Hệ Điều Hành

Người thực hiện:

Tạ Đình Tâm – CEO

Đào Huy Hoàng – Project Manager

Đồng Mạnh Hùng – Employee

Nguyễn Danh Thái – Security

Đại học Bách Khoa Hà Nội

Giảng viên hướng dẫn: TS. Phạm Đăng Hải

ONE LOVE. ONE FUTURE.

Nội dung trình bày I

1. Tổng quan dự án
2. Kiến trúc chương trình
3. Chức năng chính
4. Đánh giá và demo
5. Tài liệu tham khảo



1. Giới thiệu HDshell

HDshell là một shell đơn giản viết bằng **C++17**, phục vụ mục tiêu minh họa các cơ chế quan trọng của hệ điều hành.

Mục tiêu chính

Không thay thế Bash/Zsh, mà tập trung làm rõ logic xử lý command và vòng đời tiến trình.

- ▶ Tạo tiến trình bằng `fork()`.
- ▶ Thay thế ảnh xạ tiến trình bằng `execvp()`.
- ▶ Chờ tiến trình con bằng `waitpid()`.
- ▶ Hỗ trợ pipeline, redirection, signal và builtin command.
- ▶ Quản lý background job bằng job table riêng.

2. Yêu cầu và cách chạy

Môi trường yêu cầu

- ▶ Linux, khuyến nghị Ubuntu hoặc Docker.
- ▶ Trình biên dịch hỗ trợ C++17, ví dụ g++.
- ▶ Công cụ make.

Lệnh cơ bản

```
make
./tinysHELL
./tinysHELL path/to/script.sh
make clean
```

Docker:

```
docker buildx build -t tinysHELL-ubuntu .
docker run -it --rm tinysHELL-ubuntu
```

3. Cấu trúc thư mục

```
.
|-- Dockerfile
|-- Makefile
|-- data/
|   |-- config.json
|   '-- father.txt
|-- include/
|   |-- builtins.h
|   |-- command.h
|   |-- config.h
|   |-- executor.h
|   |-- jobs.h
|   |-- parser.h
|   '-- path_utils.h
'-- src/
    |-- builtins.cpp
    |-- command.cpp
    |-- config.cpp
    |-- executor.cpp
    |-- jobs.cpp
    |-- myShell.cpp
    '-- parser.cpp
```



4. Các thành phần chính

Nhóm xử lý command

- ▶ `myShell.cpp`: vòng lặp shell, prompt, đọc input.
- ▶ `parser.cpp`: tokenize, nhận diện pipe, redirection, background.
- ▶ `command.cpp`: thực thi `SimpleCommand` và `PipeCommand`.

Nhóm xử lý hệ thống

- ▶ `executor.cpp`: fork, `execvp`, redirection.
- ▶ `builtins.cpp`: cài đặt builtin command.
- ▶ `jobs.cpp`: job table và cập nhật trạng thái.
- ▶ `config.cpp`: đọc/ghi `data/config.json`.

5. Luồng xử lý command

1. `myShell.cpp` đọc một dòng input từ người dùng hoặc file `.sh`.
2. `Parser::parse` tách input thành token.
3. Nếu có `|`, parser tạo `PipeCommand`; ngược lại tạo `SimpleCommand`.
4. Parser xử lý các toán tử đặc biệt: `<`, `>`, `&`, `|`.
5. `Executor::executeCommand` gọi `cmd->execute()`.
6. Command tự quyết định cách chạy: builtin, external command hoặc pipeline.

Ý tưởng cốt lõi

Tách riêng **phân tích cú pháp**, **thực thi command** và **quản lý tiến trình** để code dễ mở rộng và dễ kiểm thử.

6. Builtin commands

Điều hướng và tiện ích

- ▶ `cd`: đổi thư mục hiện tại, hỗ trợ `~`, `-`, đường dẫn tuyệt đối/tương đối.
- ▶ `exit`: thoát khỏi shell.
- ▶ `clear`: xóa màn hình terminal.
- ▶ `help`: in danh sách builtin command.
- ▶ `father`: đọc file `data/father.txt`.

Tiến trình và cấu hình

- ▶ `ps`: liệt kê tiến trình từ `/proc`.
- ▶ `jobs`: liệt kê background job.
- ▶ `kill`, `killall`: gửi signal.
- ▶ `stop`, `resume`: dừng/tiếp tục tiến trình.
- ▶ `change`: cập nhật prompt config runtime.

7. External command và redirection

Luồng chạy external command

```
fork()  
child: setup redirection if needed, then execvp()  
parent: waitpid() if foreground, or add to job table if background
```

Ví dụ external command:

```
ls  
pwd  
sleep 10  
sleep 10 &
```

Ví dụ redirection:

```
cat < input.txt  
echo hello > output.txt
```

Lưu ý: builtin command cũng được redirect tạm thời rồi restore stdin/stdout của shell.

8. Pipeline

Cách hoạt động

- ▶ Parser chia input theo ký tự |.
- ▶ Mỗi stage là một SimpleCommand.
- ▶ PipeCommand::execute tạo pipe và fork từng stage.
- ▶ Exit code lấy theo command cuối cùng.

Builtin trong pipeline chạy ở child process, nên các lệnh thay đổi state như cd không ảnh hưởng tới shell cha.

Ví dụ

```
echo hello | cat
printf hello | wc -c
printf hello | wc -c | cat
cat < input.txt | wc -l > output.txt
sleep 10 | cat &
```

9. Job control và signal

Job table

Mỗi background job lưu các trường:

- ▶ job id
- ▶ pid
- ▶ status
- ▶ command

Shell gọi `Jobs::reap()` trước mỗi prompt với `waitpid(..., WNOHANG | WUNTRACED | WCONTINUED)` để cập nhật trạng thái và hạn chế zombie process.

Trạng thái job

- ▶ Running: đang chạy.
- ▶ Stopped: bị dừng bằng SIGSTOP.
- ▶ Done: đã kết thúc bình thường.
- ▶ Terminated: bị kết thúc bằng signal.

10. Config và prompt

Hai kiểu prompt

- ▶ default: hiển thị name
current/path >.
- ▶ zsh: hiển thị user@host path %
với màu riêng.

Thay đổi runtime:

```
change name MyShell  
change scheme zsh  
change color path cyan
```

```
{  
  "name": "TaTamShell",  
  "color": "default",  
  "version": "1.0",  
  "color_scheme": "zsh",  
  "prompt": {  
    "user": "green",  
    "host": "blue",  
    "path": "cyan",  
    "symbol": "magenta"  
  },  
  "history_size": 1000  
}
```

11. Ưu điểm hiện tại

- ▶ Code chia module rõ ràng: parser, command, executor, jobs, config, builtins.
- ▶ Hỗ trợ nhiều cơ chế OS cốt lõi: fork, execvp, waitpid, pipe, dup2, signal.
- ▶ Builtin command chạy đúng ngữ cảnh: command đơn chạy trong shell process, pipeline chạy trong child.
- ▶ Có job table riêng cho background jobs.
- ▶ Có xử lý zombie process bằng `waitpid(..., WNOHANG)`.
- ▶ Hỗ trợ background command, background pipeline, redirection và script `.sh` đơn giản.
- ▶ Config có thể thay đổi runtime bằng builtin `change`.

12. Điểm yếu và giới hạn

Parser và script

- ▶ Quote/escape chưa đầy đủ như Bash/Zsh.
- ▶ Chưa hỗ trợ biến môi trường \$VAR.
- ▶ Chưa hỗ trợ command substitution, wildcard expansion.
- ▶ Script chưa có if, for, while, function.

Shell feature nâng cao

- ▶ Chưa có `», 2>, 2>&1`, heredoc.
- ▶ Chưa hỗ trợ `&&, ||, ;.`
- ▶ Job control chưa có `fg, bg, tcsetpgrp`.
- ▶ Chưa có history command, autocomplete và test suite tự động.

13. Demo nhanh

```
help
clear
cd ..
ls
echo hello > /tmp/hello.txt
cat < /tmp/hello.txt
printf hello | wc -c
sleep 30 &
jobs
stop %1
resume %1
kill %1
change scheme zsh
change color path cyan
exit
```

13. Demo nhanh

Kết quả mong đợi

Người dùng quan sát được cách shell xử lý command, pipeline, redirection, background job và signal trong một chương trình C++17 nhỏ gọn.

14. Kết luận

- ▶ HDshell minh họa rõ vòng đời tiến trình từ input command đến thực thi bằng `fork/execvp`.
- ▶ Kiến trúc module giúp tách bạch parser, executor, command và job control.
- ▶ Các chức năng pipeline, redirection, background job và signal giúp mô phỏng nhiều hành vi quan trọng của shell thật.
- ▶ Dự án có thể mở rộng thêm parser nâng cao, history, autocomplete, test suite và job control đầy đủ hơn.

- [1] Nhóm phát triển HDshell, *README dự án HDshell / TinyShell*, Project 1 – Nguyên Lý Hệ Điều Hành, Đại học Bách Khoa Hà Nội.
- [2] Michael Kerrisk, *Linux man-pages project*, tham khảo các system call: fork, execvp, waitpid, pipe, dup2, kill.
- [3] Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*.

THÔNG TIN LIÊN HỆ

Project: HDshell – TinyShell

Môn học: Nguyên Lý Hệ Điều Hành

Trường: Đại học Bách Khoa Hà Nội

Giảng viên hướng dẫn: TS. Phạm Đăng Hải

Xin chân thành cảm ơn thầy và các bạn đã lắng nghe!

A decorative graphic on the left side of the slide. It features a dark blue background with a large, stylized circular pattern composed of many small red dots. The dots are arranged in concentric, slightly offset rings, creating a sense of depth and movement. The word "HUST" is centered within this pattern.

HUST

THANK YOU !

